

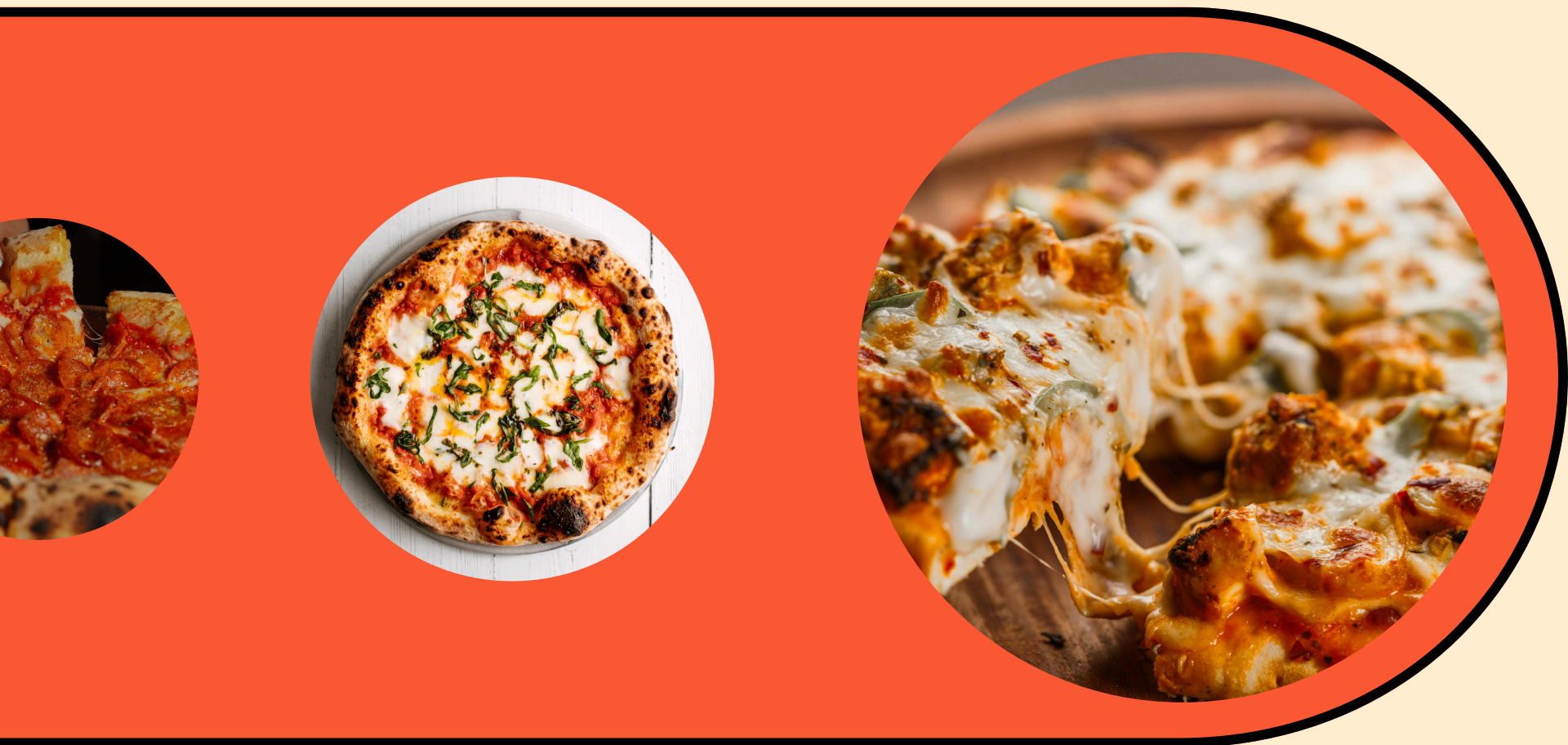
PIZZA SALES ANALYSIS PROJECT



SQL



WELCOME OUR PIZZA



OBJECTIVE

Analyze historical sales data to identify trends, forecast 15-day sales, and provide actionable insights using SQL,. Supports strategy optimization, inventory planning, and improved customer satisfaction for retail businesses.



 |     |  |    | Limit to 1000 rows  |     

```
-- 1) Retrieve the total number of orders placed
```

- `Select count(order_id) as Total_Orders_placed from orders;`

Result Grid				Filter
	Total_Orders_placed			
▶	21350			



```
2) Calculate the total revenue generated from pizza sales.  
use pizza;  
SELECT  
    ROUND(SUM(pizzas.price * order_details.quantity),  
          2) AS Total_revenue  
FROM  
    pizzas  
    JOIN  
    order_details ON pizzas.pizza_id = order_details.pizza_id;
```

Result Grid		Filter Row	
	Total_revenue		
▶	817860.05		





 |     |  |    | Limit to 1000 rows

```
1      -- 3) Calculate the total revenue generated from pizza sales.
2  ●    use pizza;
3  ●    SELECT
4          pizza_types.name, pizzas.price
5  FROM
6          pizza_types
7          JOIN
8          pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
9  ORDER BY pizzas.price DESC
10 LIMIT 1;
```

Result Grid



Filter Rows:

	name	price
▶	The Greek Pizza	35.95

-- 4) Identify the most common pizza size ordered.

- **Select** pizzas.size, count(order_details.order_details_id) **as** order_count
from pizzas **join** order_details **on**
pizzas.pizza_id = order_details.pizza_id
group by pizzas.size
order by order_count **DESC**;

Result Grid			Filter Rows:
	size	order_count	
▶	L	18526	
	M	15385	
	S	14137	
	XL	544	
	XXL	28	

```

1  -- 5) List the top 5 most ordered pizza types along with their quantities.
2  • SELECT
3      pizza_types.name, SUM(order_details.quantity) AS Sum_order
4  FROM
5      pizzas
6      JOIN
7      pizza_types ON pizzas.pizza_type_id = pizza_types.pizza_type_id
8      JOIN
9      order_details ON pizzas.pizza_id = order_details.pizza_id
10 GROUP BY pizza_types.name
11 ORDER BY Sum_order DESC
12 LIMIT 5;

```

Result Grid			Filter Rows:
	name	Sum_order	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	



```
-- 6) Join the necessary tables to find the total quantity of each pizza category ordered.
• Select pizza_types.category, sum(order_details.quantity) as Quantity
from pizzas join pizza_types on
pizzas.pizza_type_id = pizza_types.pizza_type_id
join order_details on
pizzas.pizza_id = order_details.pizza_id
group by pizza_types.category
order by Quantity desc ;
```

Result Grid			Filter Rows:
	category	Quantity	
▶	Classic	14888	
	Supreme	11987	
	Veggie	11649	
	Chicken	11050	



```
1 -- 7) Determine the distribution of orders by hour of the day.  
2 • Select hour(order_time) As hour, count(order_id) As orders from orders  
3 group by hour(order_time);
```

	hour	orders
▶	11	1231
	12	2520
	13	2455
	14	1472
	15	1468
	16	1920
	17	2336

hour	orders
17	2336
18	2399
19	2009
20	1642
21	1198
22	663



```
-- 8) Join relevant tables to find the category-wise distribution of pizzas.  
select category, count(name) from pizza_types  
group by category;
```

Result Grid			Filter R
	category	count(name)	
▶	Chicken	6	
	Classic	8	
	Supreme	9	
	Veggie	9	



```

2 • SELECT
3     pizza_types.name,
4     SUM(pizzas.price * order_details.quantity) AS Revenue
5 FROM
6     pizzas
7     JOIN
8     order_details ON pizzas.pizza_id = order_details.pizza_id
9     JOIN
10    pizza_types ON pizzas.pizza_type_id = pizza_types.pizza_type_id
11 GROUP BY pizza_types.name
12 ORDER BY Revenue DESC

```

Result Grid			Filter Rows:
	name	Revenue	
▶	The Thai Chicken Pizza	43434.25	
	The Barbecue Chicken Pizza	42768	
	The California Chicken Pizza	41409.5	





THANK YOU